

Position: Where Byte-Level Checkpoint Deduplication Dies: A Boundary Study of Semantic Redundancy in AI Training Checkpoints

Anonymous Author(s)

Abstract

Semantic checkpoint-deduplication systems report 6–896× storage savings by reasoning about *tensors*: identifying replicated, architecturally shared, or slowly changing parameters across ranks, jobs, and training steps. All of them require framework cooperation: offline tensor analysis, specialised grouped-I/O APIs, or accelerator-offloaded hashing. It is tempting to assume that an unmodified POSIX filesystem could capture the same savings with transparent, content-defined block deduplication. We test this assumption with one fixed measuring rod—BLAKE2b-128 fingerprints over fixed-size blocks—applied to real (CPU) GPT-2 small training runs in three checkpoint formats, precision and save-gap controls, synthetic multi-rank and multi-job fixtures, and, as positive controls, container image layers and multi-version source archives. The boundary is sharp: the byte layer sees the **copy axis** (DDP rank replicas, shared LoRA base files) at 100% hit ratio, but sees essentially **nothing** on the **similarity axis**—cross-step temporal redundancy ($\leq 0.29\%$ of 1 MB chunks, all zero blocks), same-architecture jobs ($\sim 0\%$; even from a shared byte-identical pretrained init, divergence takes one step), and disjoint parameter shards (0%)—even though a general compressor fares no better (zstd $1.08\times$; XOR-delta $\leq 1.28\times$). The same code path scores 59–98% on image layers and source trees. We argue the right role for a transparent layer is *learn-to-skip*: classify workloads before touching bytes, with asymmetric loss that can only ever affect performance, never correctness.

ACM Reference Format:

Anonymous Author(s). 2027. Position: Where Byte-Level Checkpoint Deduplication Dies: A Boundary Study of Semantic Redundancy in AI Training Checkpoints. In *Proceedings of 19th ACM Workshop on Hot Topics in Storage and File Systems (HotStorage'27)*. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

HotStorage'27, TBD

2027. ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

Checkpoints already dominate AI-cluster storage traffic. Two FAST'26 systems attack the waste semantically: AdaCheck [10] classifies checkpoint tensors as replicated, architecturally shared, or temporally similar, and reports 6–896× savings; AITURBO [7] batches tensor I/O into grouped operations and computes BLAKE3 [13] fingerprints on XPU kernels because CPU hashing cannot keep up with the storage fabric. Both require cooperation: the application must expose tensor identity or use a specialised API. The closest byte-level predecessor, Kaiser et al. [9], characterised block-deduplication potential in *HPC* checkpoints and found application-dependent but real recurring savings. Ours do not behave the same.

Transparent block deduplication is a textbook POSIX-filesystem feature [15, 17]: fingerprint content blocks, suppress redundant writes, and leave the application untouched. If semantic checkpoint redundancy were visible at the byte layer, *unmodified* PyTorch jobs would benefit simply by mounting the filesystem—no framework changes, no XPU. We asked exactly one question: *of the redundancy that tensor-level systems exploit, how much reaches an unmodified POSIX byte layer?*

We answer with a controlled measurement on a self-contained, reproducible GPT-2 small (124 M parameter) training fixture—real forward/backward and AdamW updates, not synthesised tensors—plus axis-specific fixtures and control workloads where byte deduplication is *expected* to win.

Findings. Cross-step identical 1 MB blocks are 0.14–0.28% for full checkpoints and **0%** for weights-only files; every hit is a zero block. Alignment sweeps, precision, save gaps, zstd, and XOR-delta do not help. Yet identical DDP rank replicas and shared LoRA bases match at 100%, and image layers/source archives score 59–98% under the same code. Redundancy splits by **axis**, not by workload.

Contributions.

- (1) A controlled, reproducible study of redundancy visibility at the POSIX byte layer across three formats, two precisions, three save gaps, and all three redundancy axes.

Table 1: Redundancy axes exploited by semantic checkpoint systems.

Axis	Example	Who benefits today
A. Intra-job copies	DDP replicates all parameters/optimizer state per rank; TP replicates embeddings	Any byte-identical copy mechanism
B. Inter-job similar-ity	Same architecture, different data/seed; LoRA jobs share one frozen base	Tensor-keyed systems [10]
C. Inter-step similar-ity	Consecutive checkpoints of one job	Tensor diff / grouped-I/O [7, 10]

- (2) A mechanism explanation—dense gradients plus decoupled weight decay make every fp32 byte change every step—with controls ruling out chunking, alignment, precision, and temporal distance.
- (3) A sharp copy/similarity boundary and its design implication: transparent storage should *learn to skip* hashing under an asymmetric-loss contract that keeps ML out of the correctness path.

2 Background and Redundancy Taxonomy

Checkpoint layouts. `torch.save` serialises a pickle object inside an uncompressed (STORED) zip container; tensor storage lands in numbered `data/N` entries. `torch.distributed.checkpoint` (DCP) writes one or more `.distcp` shard files plus metadata in the same zip-based format. `safetensors` [8] instead stores tensors as one contiguous little-endian byte region preceded by a JSON header. None of these formats compresses fp32 state, because it does not compress (§4).

Byte-level deduplication. A transparent client fingerprints fixed-size blocks (we use BLAKE2b-128) and suppresses blocks whose fingerprints already exist. Cost is dominated by hashing—why AITURBO computes BLAKE3 [13] on XPU kernels [7]—and by alignment: fixed grids miss offset-drifted content, which content-defined chunking [12, 18] fixes at extra CPU cost. Block stores from Venti [15] to iDedup [17] follow this model; iDedup also documented fragmentation cost on reads.

Three redundancy axes. Semantic systems exploit three structurally different sources of repeated bytes:

The crucial distinction is **copies versus similarities**. A copy is the same tensor placed in several files; the bytes are identical. A similarity is *the same parameter* in two semantic versions—updated by an optimizer, or initialised from a different seed. Semantic tensor keys (name, shape, allocation identity) recognise the latter; byte fingerprints can only match exact content. Table 1 is the taxonomy; Table 3 (§4.1) fills in what the byte layer actually sees on each axis.

3 Methodology

Fixture (inter-step axis, C). We implement a self-contained GPT-2 small (50,257 vocab, 1,024 positions, 768 width, 12 heads, 12 layers; tied input/output embeddings; pre-LayerNorm; HF-compatible tensor naming) and train it with AdamW (lr 10^{-3} ; betas (0.9, 0.999); eps 10^{-8} ; weight decay 0.01) on random token streams (batch 1, sequence 128) for 20 steps, saving every step in three layouts: full `torch.save` (1,493 MB), weights-only (498 MB), and DCP (1,494 MB)—about 65 GB total. A 100-step run provides long-gap and precision controls (torch 2.4.1+cpu, seed 20260910; every script is checked in and re-runnable). We use a small CPU model because the mechanism under test (element-wise optimizer updates) is independent of parameter count.

Analyzer. For every checkpoint we fingerprint fixed blocks of {4 KB, 64 KB, 1 MB, 4 MB} using BLAKE2b-128, in a raw *file* view and a *zipdata* view where every `data/N` entry is chunked independently. We measure hit ratio against the previous version (*hit-prev*) and the union of all earlier versions (*hit-any*), all-pairs Jaccard, and offset-shift sweeps that re-chunk the reference after skipping {1, 16, 256, 4096} bytes—our detector for “identical content, drifted offset”, the condition CDC would rescue; a synthetic fixture validates exact instance counts.

Axis fixtures (A and B). From two independent 3-step runs we materialise four byte-identical full-checkpoint *rank replicas* per format (md5-verified), four disjoint 1/4 parameter *shards*, same-architecture cross-job pairs, and two LoRA-style jobs sharing one identical 498 MB base file with distinct 0.6 MB adapters. A third run reuses job A’s byte-identical init (determinism-asserted) with a different data stream for 6 weights-only steps, modelling a fine-tuning fleet that shares one pretrained base.

Positive controls. Three images built sequentially on ubuntu:20.04, exported as flat rootfs tarballs and `docker save` OCI archives, plus `git archive` tarballs of five commits spanning ~276, 60, 60, and 20 commits. The image *application* layers are deliberately **full sibling layers**, not an optimised incremental chain. For archives

Table 2: Steady-state hit-any ratios over steps 5–19. Every hit is a zero block.

Layout / view	4 KB	64 KB	1 MB	4 MB
torch.save / file	0.0037	0.0036	0.0014	0
torch.save / zipdata	0.0037	0.0037	0.0029	0
weights-only / file	0	0	0	0
DCP / file	0.0037	0.0036	0.0028	0

we measure both a *per-file* view (each tar member chunked from offset zero) and the naive *raw* tar-stream view.

Scope. This is a controlled microbenchmark, not field-trace analysis (§7).

4 Findings

4.1 Temporal redundancy is invisible at the byte layer

Every hit in the full-checkpoint rows (Table 2, Figure 1) is an all-zero block: exact-block ratio equals zero-block ratio (0.37%), and weights-only files score exactly zero at every size. At 1 MB, all-pairs Jaccard across steps 0–19 is a flat 0.000351 regardless of step distance. Temporal similarity—what AdaCheck and AITURBO harvest step after step—does not exist as equal bytes. Notably, AITURBO’s finest detection granularity is 4 MB [7]; at exactly that setting every format yields zero cross-step hits—XPU hashing would be spent on a stream containing nothing to find.

Not an alignment problem. Two complementary tests. *Within* checkpoints, shifting the previous checkpoint’s chunk grid by 1, 16, or 256 bytes leaves the hit ratio unchanged (e.g. 0.001404 at 1 MB for every shift; a 4096 shift removes the prefix chunk and nothing else). Identical-but-drifting content, the failure mode content-defined chunking fixes, is absent: the hit-any ratio is flat across all four chunk sizes (Table 2), leaving no hidden redundancy for CDC to uncover.

Across workload families, alignment matters sharply. Chunking archive byte streams naively (raw tar view) collapses even our positive controls: rootfs 4 KB hits drop 58.6%→3.9% (v2) and 97.7%→10.4% (v3); source archives drop 94.8%→54.3% in the closest pair and to 1.4–4.5% at moderate distances; at 1 MB every raw-stream pair is zero, as tar reordering and headers destroy grid alignment. Checkpoint zipdata entries, in contrast, *are* chunked from independent aligned boundaries—and still score zero. Alignment is necessary (wrong boundaries read zero) but not sufficient.

Not precision, age, or compressibility. *Precision control.* bf16 weights across 100 steps yield 6–11 matching

4 KB blocks out of 60,726 ($\leq 0.018\%$) and zero at 64 KB or 1 MB; fp32 weights are exactly zero at gaps of 1, 50, and 100 steps. Lower mantissa precision does not quantise Adam updates into byte-identical blocks.

Format control. A safetensors run replaying the **identical** 20-step sequence (bit-identical losses: 10.989, 10.916, 10.968, ...) scores 0.0025% hits at 4 KB and zero above—the result is a property of the bytes, not the zip container.

Compression control. Perhaps redundancy exists in a form blocks cannot express. zstd-1 on one checkpoint compresses 1.08 $\times$; an XOR-delta stream between adjacent steps compresses only 1.15–1.28 $\times$ under zstd; byte-wise equality between adjacent steps is just 19.8–30.3%. fp32 training state in motion carries almost no extractable redundancy by any measure.

Mechanism: every byte is touched every step.

The zero result has a precise mechanism. (1) AdamW applies decoupled weight decay: every participating element receives $p \leftarrow p - lr \cdot wd \cdot p$ each step, a relative change of 10^{-5} —small but guaranteed to flip fp32 low bytes. (2) Adam momentum slots decay and accumulate per element. (3) In our tied-head GPT-2, the lm-head backward produces a dense gradient through the whole embedding matrix, and the embedding backward materialises **zero (not None) gradients** for unsampled rows, so weight decay touches those rows too—we verified that rows unused by the current batch still change bytes every step; and (4) *the churn survives learning-rate decay*: an element’s per-step relative change is $lr \cdot wd$ (weight decay) or $lr \cdot |\hat{m}|/\sqrt{\hat{v}}/|p|$ (Adam step), whose ratio $|\hat{m}|/\sqrt{\hat{v}}$ is scale-free; for $|p| \sim 10^{-2}$ the Adam term is $10\text{--}50 \times lr$, and fp32’s unit roundoff 6×10^{-8} keeps bytes flipping while $lr > 10^{-9}$ —far below any decay tail. Weight decay alone would freeze at $lr < 6 \times 10^{-6}$; the gradient step is the load-bearing term, so freezing requires gradients to vanish or become **None**. At fp32 essentially every parameter byte changes each step, while tensor *keys* (names, shapes, identity) stay constant: the axis boundary in one sentence.

Gradient density sets the boundary. The mechanism assumes every parameter receives a gradient each step—true for dense-update training like our fixture, and the reason its bytes churn. PyTorch-style optimisers apply *no* update to a parameter whose gradient is **None**, skipping even decoupled weight decay, so such bytes stay exactly stable. Sparsely-updated regimes—routed-out MoE experts, frozen stages, sparse schedules—sit closer to the copy axis and are byte-deduplicable, exactly the labelled positives a skip gate must not lose. Gradient-density proxies are observable from the I/O stream alone, making them natural gate features.

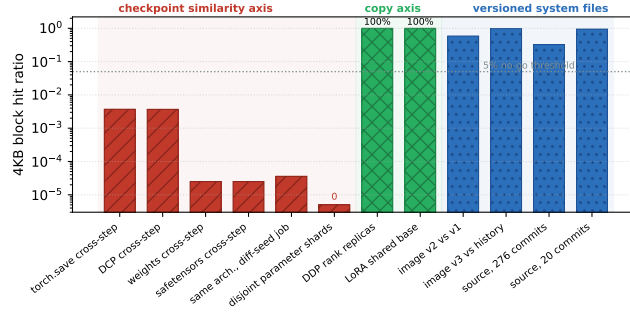


Figure 1: Cross-step hit-any ratio per checkpoint format and chunk size (log scale). The byte layer finds only zero blocks on training checkpoints (red), versus 59–98% hits for container image layers and source archives.

Table 3: Axis map (per-file aligned blocks, same trained states).

Axis	Comparison	4 KB	64 KB	1 MB
A	DDP rank replicas	1.000	1.000	1.000
copies	(torch.save/safetensors)			
A	Disjoint FSDP/TP	0	0	0
divides	shards vs earlier shards			
B cross-	full ckpt / weights	0.000	0.000	0.0007
job	safetensors, diff. seed			
B	shared init, 1 step apart	0.006	0.005	0.004
co-init	(weights)			
B	shared base file, job2 vs	1.000	1.000	1.000
LoRA	job1			
B	distinct adapter files	0	0	—
LoRA				
B	whole-job directories	0.9988	0.9988	1.000
LoRA				
C tem-	consecutive steps (Ta-	0.0037*	0.0036*	0–0.0029
poral	ble 2)			

*hits are zero blocks. The single 1 MB inter-job hit (1 of 1,423 blocks) is zero content, not structure. The co-init residual is the zero-gradient positional-embedding region, where both jobs trace byte-identical weight-decay-only trajectories.

Redundancy-axis map: copies visible, similarities invisible.

Whole-object copies—rank replicas and shared base files—collapse to one physical copy transparently; the LoRA directory result (99.9%) shows a transparent store already handles the dominant bytes of the LoRA pattern, missing only the 0.6 MB adapters. Everything requiring recognition of “the same parameter in another version”—consecutive steps, independent jobs, disjoint shards—is invisible. The 6–896× headline savings of semantic systems live on this invisible side; their framework requirements are forced by where the redundancy is. Sharing a pretrained base does not change the verdict: starting two jobs from a byte-identical init, a single step on different data collapses block identity to a 0.4–0.6% residual (the zero-gradient positional-embedding region, Table 3).

Table 4: Positive controls (per-file view, hit-any).

Workload	4 KB	64 KB	1 MB
Image rootfs v2 vs v1	0.586	0.623	0.660
Image rootfs v3 vs v1+v2	0.977	0.974	0.981
Source, ~276 commits	0.324	0.115	—
apart			
Source, moderate gaps	0.705/0.754	0.563/0.585	—
Source, ~20 commits apart	0.948	0.956	—
Training checkpoints (all formats)	0	0	0

The divergence horizon of the dominant fine-tuning fleet pattern is *one step*.

Positive controls: the same instrument scores 59–98%.

The 75 MB ubuntu base blob is shared verbatim across all three images; even the 191/197 MB sibling application layers—full reinstalls—contain 59% and 98% of blocks seen earlier, because dpkg overwrites the same library and binary files with identical content (Table 4); the same holds at all block sizes, while source archives decay smoothly with commit distance. System files are byte-stable across versions; optimizer state is byte-stable for exactly one step. With the same code scoring 98% and 0%, the zero cannot be blamed on the instrument.

5 Implications: Learn to Skip, Not to Deduplicate

A classifier before the hash. If hashing is expensive (AITURBO’s XPU is the admission [7]) and temporal checkpoints never hit, a transparent client should not be smarter about *how* it hashes checkpoint streams—it should decide *whether to hash at all* from signals available before touching bytes: write pattern, file size/age, directory topology (rank replicas), and cheap prefix probes.

Copy-heavy paths always hash; churn-heavy training state skips to write-through.

Asymmetric loss keeps ML off the correctness path. A gating classifier makes one mistake in each direction: a false positive (hashed a unique stream) wastes fingerprints; a false negative (skipped a redundant stream) loses one saving. Both affect performance only. Suppression still requires an exact fingerprint match, so ML can neither corrupt data nor lose a byte—unlike prediction-in-the-data-path systems such as LinnOS [6] or KML’s learned kernel heuristics [2]: prediction proposes, exact matching disposes. Our measurements supply the labelled **negative** workloads such a gate previously lacked.

Why learned rather than rule-based. The negative class we measure is simple—periodic multi-GB rewrites of one path—and a three-rule gate (size, period, cadence) would classify today’s fixtures. The rule set, however, is format-relative: sharded DCP layouts, weights-only saves, quantised optimizer states, and sparse-update MoE training (§4) shift the boundary in ways a fixed rule cannot anticipate. A gate retrained on labelled positives and negatives tracks that drift, and the asymmetric loss makes its mistakes cheap. Placement: per file, in the dedup client, before the first fingerprint—late enough that exact matching owns correctness, early enough that no hash cost is paid.

A lower bound for semantic systems. The zero on the similarity axis quantifies the *necessity* of tensor cooperation: no transparent cleverness—CDC, larger blocks, compression—can intercept savings whose identical units are tensor keys, not bytes. The 100% on the copy axis shows transparent dedup remains right for replicated saves and shared bases; a deployed system should compose both.

Existence proof. In our research filesystem—a log-structured, needle-based object store with an inline kernel dedup hot path—the write path consults the dedup index only when a cheap client-side gate deems a write likely-unique, and appends directly otherwise. Replaying four 256 MB sequential write rounds, per-round Write-Needle (dedup metadata) RPCs drop from 1,020 to 258 and steady-state write bandwidth rises 50%, with md5-identical stored layout. The gated path is cheap and safe; this paper determines *where it must not be switched on*. (System name withheld for blind review.)

6 Related Work

Semantic checkpoint storage. AdaCheck [10] rewrites tensors across parallelism, architecture, and temporal dimensions; AITURBO [7] adds grouped I/O and XPU

BLAKE3 fingerprinting. Multilevel checkpointing systems for HPC (Moody et al. [11], FTI [3]) optimise snapshot tiering and recovery interval, not cross-run byte sharing. Kaiser et al. [9], the closest predecessor, measured byte-level deduplication potential across HPC checkpoints, where static memory regions produced savings that vary by application. We revisit the question for AI-training state under AdamW, splitting redundancy into copy and similarity axes: the answer flips for temporal similarity but holds for copies.

Writing less instead of deduplicating. Checkpoint bytes can be shrunk at the source: tiered multilevel snapshotting [3, 11], differential checkpointing (CheckN-Run [5]), quantised optimizer state (8-bit blockwise Adam [4]; ZeRO partitioning/offload [16]), and PyTorch DCP [1], whose per-rank disjoint shards remove axis A outright. All are framework-visible changes; our question is what remains for the layer that changes nothing.

Block deduplication and chunking. Venti [15] established content-addressed blocks; iDedup [17] studied inline deduplication and its fragmentation cost; LBFS [12] introduced and FastCDC [18] optimised content-defined chunking. Our shift sweep tests CDC’s precondition (offset-drifted identical content) directly and finds it absent.

Learning in storage. LinnOS [6] reroutes I/Os from a learned latency model; KML [2] sets kernel heuristics such as readahead with learned models. In both, model output directly changes the I/O action. Our gate differs structurally: ML decides only whether an exact algorithm runs, never what data it returns. Mooncake [14] targets KVCACHE-centric serving traffic, a separate track we do not enter; traditional-workload baselines are deferred to the full-paper version.

7 Limitations and Conclusion

Limitations. One architecture (124M GPT-2), one optimiser family (AdamW), random-token CPU training, single-process fixtures: absolute numbers will move at scale, but the mechanism—per-element updates flip fp32 bytes; similarity keys are tensor names—is architecture-independent. Long-horizon lr-schedule runs are argued analytically; multi-node traces, sparse-update and non-AdamW regimes, a full CDC implementation, and restore-side fragmentation are clear extensions.

Conclusion. The byte layer sees every *copy* (100%) and none of the *similarity* semantic systems monetise, while the same code scores 59–98% on images and sources. Transparent storage cannot earn that 6–896×; it should learn to skip churn and reserve exact fingerprinting for where copies live.

References

- [1] 2024. PyTorch Distributed Checkpoint (DCP). <https://pytorch.org/docs/stable/distributed.checkpoint.html>. Accessed September 2026.
- [2] Ibrahim Umit Akgun, Ali Selman Aydin, Ahmad Shaikh, Lukas Velikov, and Erez Zadok. 2021. A Machine Learning Framework to Improve Storage System Performance. In *13th ACM Workshop on Hot Topics in Storage and File Systems (HotStorage '21)*. ACM. doi:10.1145/3465332.3470875
- [3] Leonardo A. Bautista-Gomez, Stefano Tsuboi, Dimitri Komatitsch, Franck Cappello, Naoya Maruyama, and Satoshi Matsuoka. 2011. FTI: High Performance Fault Tolerance Interface for Multi-Level Checkpoints. In *17th International Conference on Parallel Processing (Euro-Par '11), Part II*. Springer, 1682–1693.
- [4] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. 2022. 8-Bit Optimizers via Block-Wise Quantization. In *10th International Conference on Learning Representations (ICLR '22)*.
- [5] Assaf Eisenman, Kiran Kumar Matam, Steven Ingram, Dheevatsa Mudigere, Raghuraman Krishnamoorthi, Krishnakumar Nair, Misha Smelyanskiy, and Murali Annavaram. 2022. Check-N-Run: A Checkpointing System for Training Deep Learning Recommendation Models. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI '22)*. USENIX Association, 929–943.
- [6] Mingzhe Hao, Levent Toksoz, Nanqin Li, Edward Edberg Halim, Henry Hoffmann, and Haryadi S. Gunawi. 2020. LinOS: Predictability on Unpredictable Flash Storage with a Lightweight Neural Engine. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI '20)*. USENIX Association, 629–646.
- [7] Yingyi Hao, Ting Yao, Xingda Wei, Dingyan Zhang, Tianle Sun, Yiwen Zhang, Zhiyong Fu, Huatao Wu, and Rong Chen. 2026. Fast Cloud Storage for AI Jobs via Grouped I/O API with Transparent Read/Write Optimizations. In *24th USENIX Conference on File and Storage Technologies (FAST '26)*. USENIX Association. System name: AITURBO.
- [8] Hugging Face. 2024. safetensors Serialization Format. Software specification. <https://huggingface.co/docs/safetensors>
- [9] Jürgen Kaiser, Ramy Gad, Tim Süß, Federico Padua, Lars Nagel, and André Brinkmann. 2016. Deduplication Potential of HPC Applications' Checkpoints. In *2016 IEEE International Conference on Cluster Computing (CLUSTER '16)*. IEEE, 413–422. doi:10.1109/CLUSTER.2016.32
- [10] Weijie Liu, Shengwei Li, Zhiquan Lai, Keshi Ge, Qiaoling Chen, Peng Sun, Dongsheng Li, and Kai Lu. 2026. AdaCheck: An Adaptive Checkpointing System for Efficient LLM Training with Redundancy Utilization. In *24th USENIX Conference on File and Storage Technologies (FAST '26)*. USENIX Association, 271–289.
- [11] Adam Moody, Greg Bronevetsky, Kathryn Mohror, and Bronis R. de Supinski. 2010. Design, Modeling, and Evaluation of a Scalable Multi-Level Checkpointing System. In *International Conference for High Performance Computing, Networking, Storage and Analysis (SC '10)*. IEEE Computer Society.
- [12] Athicha Muthitacharoen, Benjie Chen, and David Mazières. 2001. A Low-Bandwidth Network File System. In *Proceedings of the 18th ACM Symposium on Operating Systems Principles (SOSP '01)*. ACM, 174–187. doi:10.1145/502034.502052
- [13] Jack O'Connor, Jean-Philippe Aumasson, Samuel Neves, and Zooko Wilcox-O'Hearn. 2020. BLAKE3: One Function, Fast Everywhere. Specification and reference implementation. <https://blake3.io>
- [14] Ruoyu Qin, Zheming Li, Weiran He, Jiale Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2025. Mooncake: Trading More Storage for Less Computation—A KVCache-centric Architecture for Serving LLM Chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST '25)*. USENIX Association, 155–170. Best Paper.
- [15] Sean Quinlan and Sean Dorward. 2002. Venti: A New Approach to Archival Storage. In *1st USENIX Conference on File and Storage Technologies (FAST '02)*. USENIX Association, 89–101.
- [16] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2020. ZeRO: Memory Optimizations Toward Training Trillion Parameter Models. In *International Conference for High Performance Computing, Networking, Storage and Analysis (SC '20)*. IEEE Computer Society.
- [17] Kiran Srinivasan, Tim Bisson, Garth R. Goodson, and Kaladhar Voruganti. 2012. iDedup: Latency-aware, Inline Data Deduplication for Primary Storage. In *10th USENIX Conference on File and Storage Technologies (FAST '12)*. USENIX Association.
- [18] Wen Xia, Yukun Zhou, Hong Jiang, Dan Feng, Yu Hua, Yuchong Hu, Qing Liu, and Yucheng Zhang. 2020. The Design of Fast Content-Defined Chunking for Data Deduplication Based Storage Systems. *IEEE Transactions on Parallel and Distributed Systems* 31, 9 (2020), 2017–2031. doi:10.1109/TPDS.2020.2984601